

Exercise 1-Setting Up Junit

To set up JUnit in your Java project to start writing unit tests.

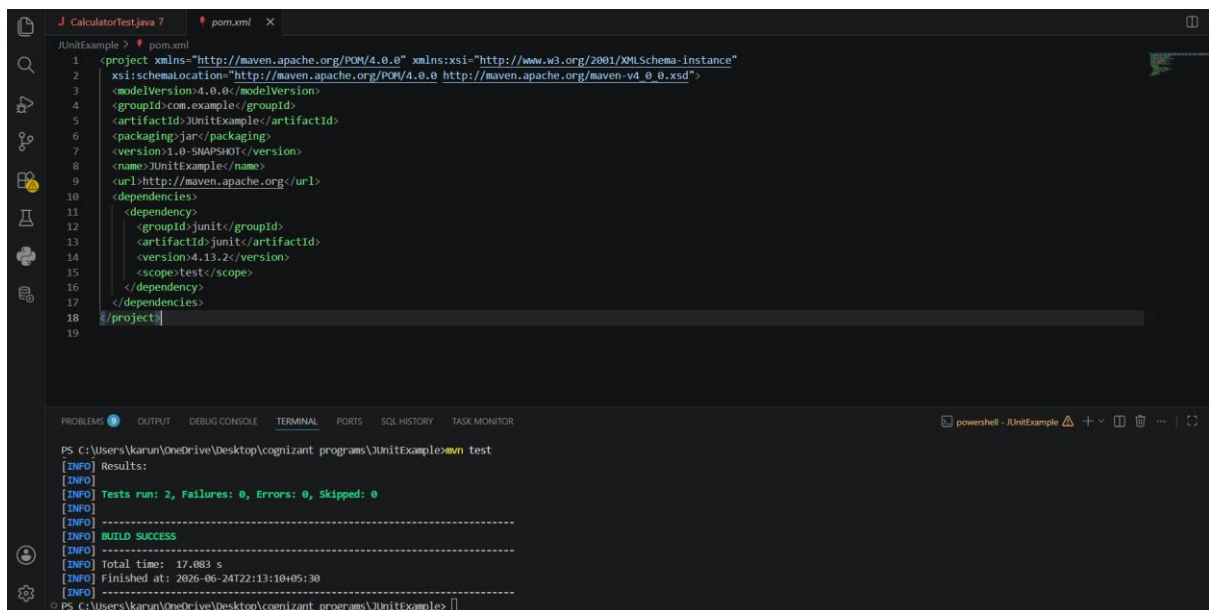
Dependencies added in pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>JUnitExample</artifactId>
  <packaging>jar</packaging>
  <version>1.0-SNAPSHOT</version>
  <name>JUnitExample</name>
  <url>http://maven.apache.org</url>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```



Exercise 3- Assertions in JUnit

To use different assertions in JUnit to validate your test results.

Code for Assertion.java

```
package com.example;

import org.junit.Test;

import static org.junit.Assert.*;

public class AssertionsTest {

    @Test
    public void testAssertions() {

        assertEquals(5, 2 + 3);

        assertTrue(5 > 3);

        assertFalse(5 < 3);

        assertNull(null);

        assertNotNull(new Object());
```



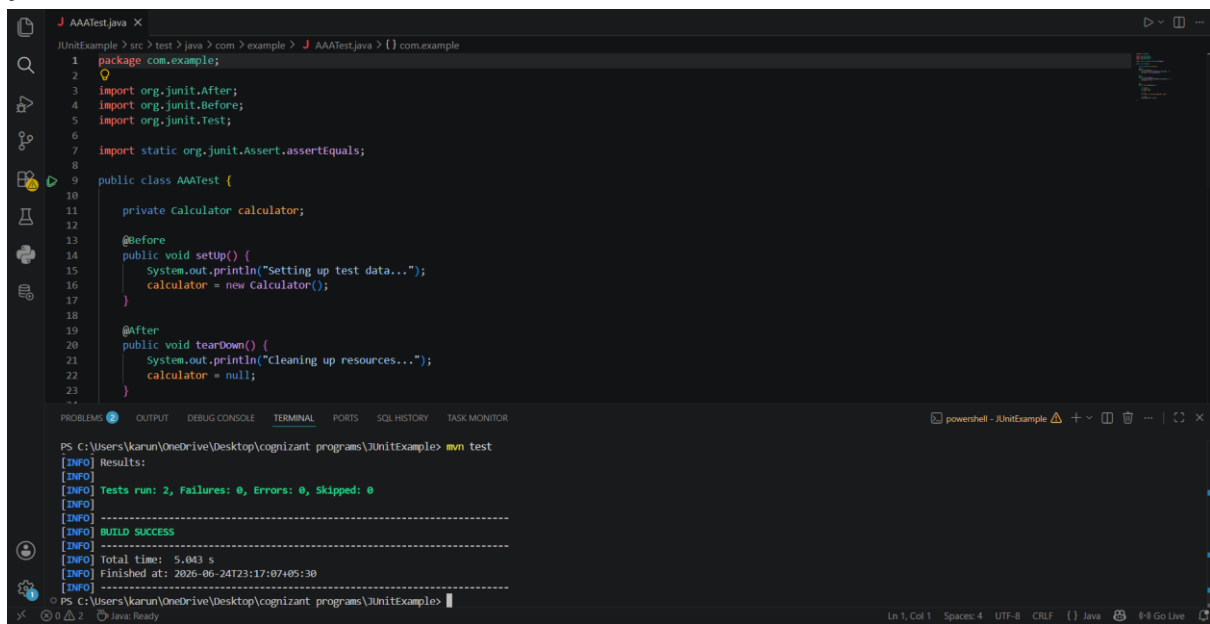
```
private Calculator calculator;

@Before
public void setUp() {
    System.out.println("Setting up test data...");
    calculator = new Calculator();
}

@After
public void tearDown() {
    System.out.println("Cleaning up resources...");
    calculator = null;
}

@Test
public void testAddition() {
    int num1 = 10;
    int num2 = 20;
    int result = calculator.add(num1, num2);
    assertEquals(30, result);
}
```

}



The screenshot shows an IDE with a Java file named `AAAtest.java` and its execution results in the terminal. The Java code is as follows:

```
1 package com.example;
2
3 import org.junit.After;
4 import org.junit.Before;
5 import org.junit.Test;
6
7 import static org.junit.Assert.assertEquals;
8
9 public class AAAtest {
10
11     private Calculator calculator;
12
13     @Before
14     public void setup() {
15         System.out.println("Setting up test data...");
16         calculator = new Calculator();
17     }
18
19     @After
20     public void tearDown() {
21         System.out.println("Cleaning up resources...");
22         calculator = null;
23     }
24 }
```

The terminal output shows the results of running the tests:

```
PS C:\Users\karun\OneDrive\Desktop\cognizant programs\JUnitExample> mvn test
[INFO] Results:
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 5.043 s
[INFO] Finished at: 2026-06-24T23:17:07+05:30
[INFO]
PS C:\Users\karun\OneDrive\Desktop\cognizant programs\JUnitExample>
```

The IDE interface includes a sidebar with icons for Explorer, Search, Run and Debug, Source Control, Test Explorer, and Extensions. The bottom status bar shows the current file is `Ln 1, Col 1`, with 4 spaces, UTF-8 encoding, CRLF line endings, and the language is set to Java.